



MILESTONE 4

Gold Star Schema

Reshaping Silver into a business-ready star schema with dbt, tested and version-controlled



STEP 1

Set Up dbt and the Warehouse

WHAT

Created a Serverless SQL Warehouse (atliq-dbt-wh, 2X-Small, auto-stop 10 min) as the dbt compute target, generated a Databricks access token, then installed dbt-databricks in an isolated Python virtual environment.

WHY

A Serverless SQL Warehouse is the standard dbt target and sidesteps the VM-quota issue from M3. An isolated venv keeps dbt's dependencies from clashing with other tools.

1

STEP 1 · SCREENSHOT 1 of 2

Set Up dbt and the Warehouse

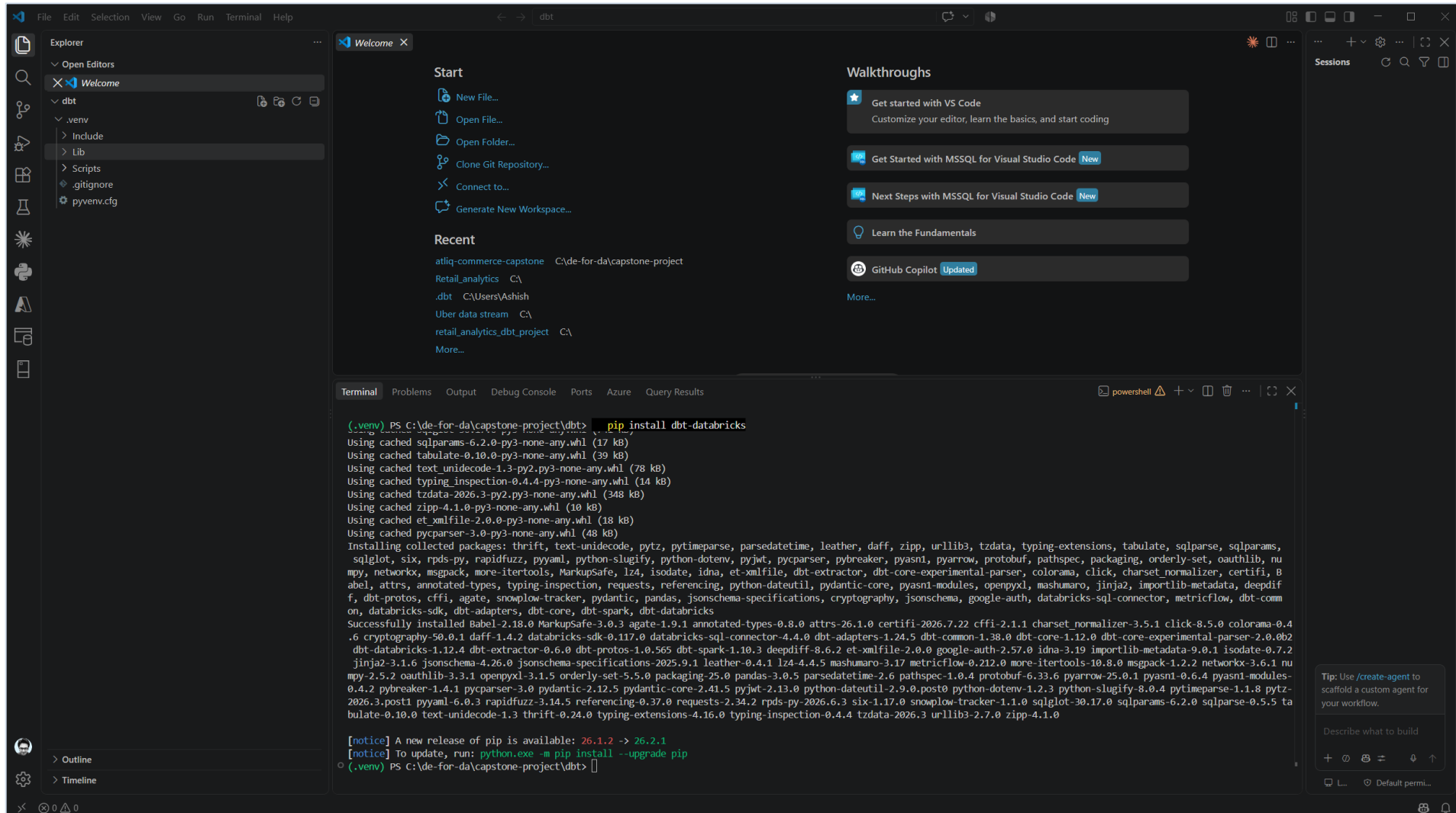
The screenshot shows the Databricks web interface in a browser. The URL is `https://adb-7405614391071920.0.azuredatabricks.net/sql/warehouses/28d7b90aacdb4c8/connectionDetails?o=7405614391071920`. The page title is "atliq-dbt-wh". The left sidebar contains navigation options: Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL (SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses), Data Engineering (Runs, Data Ingestion, Visual Data Prep), AI/ML (Playground, Agents, AI Gateway, Experiments, Features, Models, Serving). The main content area shows the "Connection details" tab for the warehouse. It includes a "New" button, a search bar, and a "Start" button. Below the title, there are tabs for Overview, Connection details (active), and Monitoring. A message says "Use these details to connect to this warehouse" with a link to "Create a personal access token". A row of icons for various tools (Tableau, Power BI, dbt, Python, Java, Node.js, Go, More tools) is displayed. The connection details section contains the following fields:

- Server hostname:** `adb-7405614391071920.0.azuredatabricks.net`
- Workspace ID:** `7405614391071920`
- HTTP path:** `/sql/1.0/warehouses/28d7b90aacdb4c8`
- JDBC URL:** `jdbc:databricks://adb-7405614391071920.0.azuredatabricks.net:443/default;transportMode=http;ssl=1;AuthMech=3;httpPath=/sql/1.0/warehouses/28d7b90aacdb4c8;`
- OAuth URL:** `https://adb-7405614391071920.0.azuredatabricks.net/oidc`

At the bottom of the connection details section, it states: "Databricks supports drivers released within the last two years. [Download drivers here](#)".

atliq-dbt-wh connection details: server hostname and HTTP path used by dbt

Set Up dbt and the Warehouse



pip install dbt-databricks inside the project virtual environment



STEP 2

Scaffold the Project (dbt init)

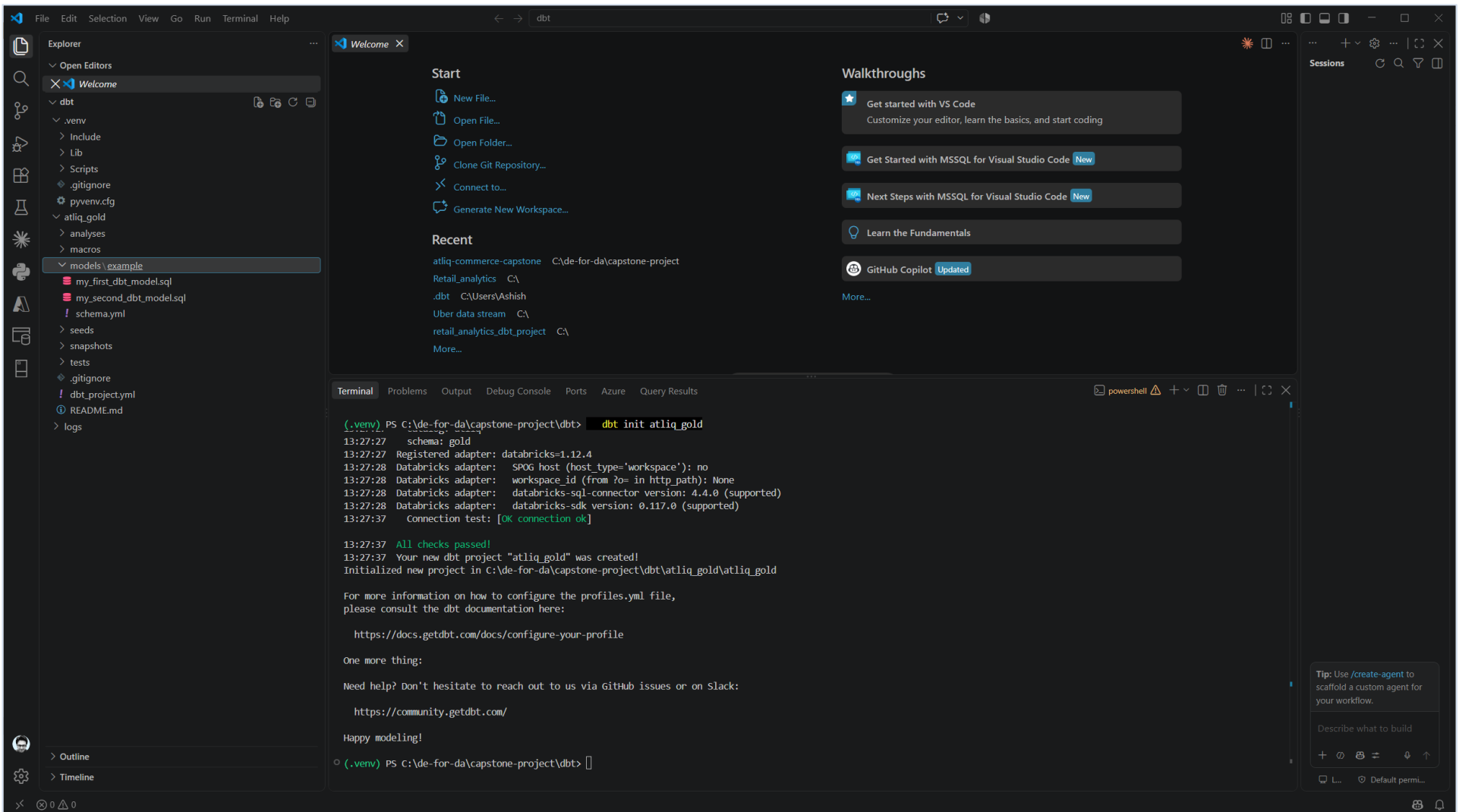
WHAT

Ran `dbt init atliq_gold` to scaffold a fresh project, selecting the Databricks adapter and Unity Catalog, with catalog `atliq` and schema `gold`. Then removed the demo example models.

WHY

dbt init generates the standard project skeleton (`dbt_project.yml`, `models` folder, `config`). Starting from the `real scaffold` command makes the project genuinely mine to build up.

Scaffold the Project (dbt init)



dbt init atliq_gold: connection test OK, All checks passed



STEP 3

Author the Staging Layer

WHAT

Wrote 6 staging models (stg_customers, stg_products, stg_orders, stg_order_items, stg_payments, stg_supplier_price_list), each a thin view over its Silver source via source(), plus sources.yml declaring the Silver tables.

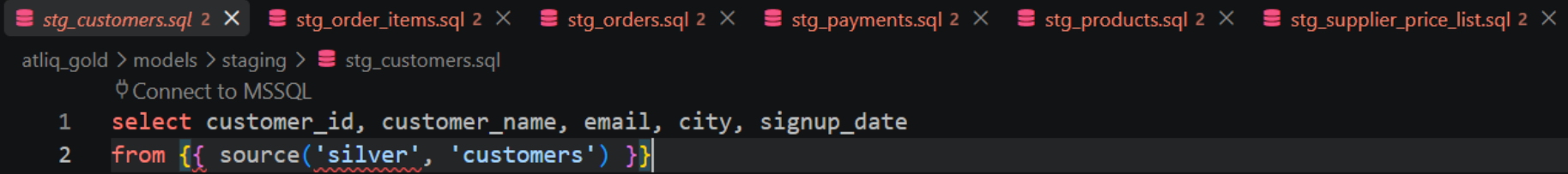
WHY

Staging is a stable interface over Silver: Gold models reference stg_ models, not raw tables, so a source change is fixed in one place. Views keep them zero-copy, and dbt traces lineage.

3

STEP 3 • SCREENSHOT

Author the Staging Layer



The screenshot shows a SQL editor interface with a dark theme. At the top, there are several tabs for different SQL files: `stg_customers.sql`, `stg_order_items.sql`, `stg_orders.sql`, `stg_payments.sql`, `stg_products.sql`, and `stg_supplier_price_list.sql`. The active tab is `stg_customers.sql`. The editor content shows the following SQL code:

```
atliq_gold > models > staging > stg_customers.sql
ψ Connect to MSSQL
1 select customer_id, customer_name, email, city, signup_date
2 from {{ source('silver', 'customers') }}
```

Six stg_ models; stg_customers reads source('silver', 'customers')



STEP 4

Build the Dimensions

WHAT

Wrote `dim_customer` (with `signup_month` cohort), `dim_product` (joining supplier cost and computing `unit_margin`), and `dim_date` (a generated 2024-2026 calendar spine).

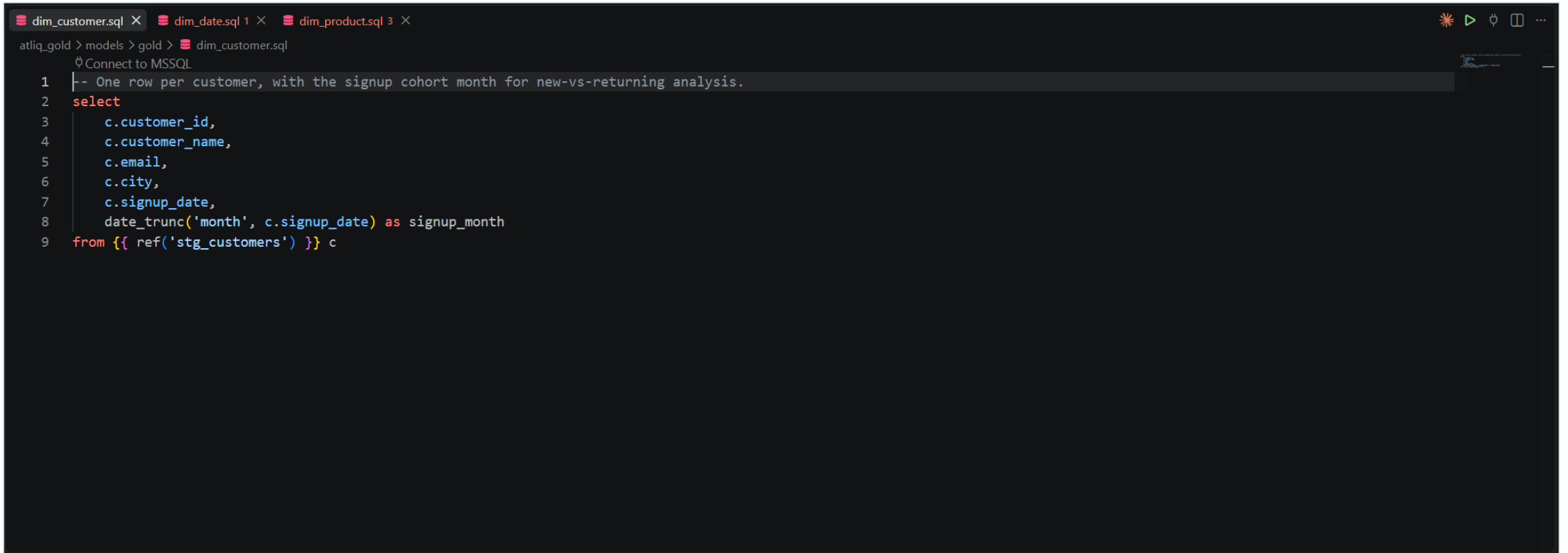
WHY

Dimensions hold descriptive context. `dim_product` combines two sources into a margin attribute; `dim_date` lets reports slice by month, quarter, year, and weekday without recomputing.

4

STEP 4 • SCREENSHOT 1 of 3

Build the Dimensions

A screenshot of a SQL editor interface. At the top, there are three tabs: 'dim_customer.sql', 'dim_date.sql 1', and 'dim_product.sql 3'. The main editor area shows a SQL query for 'dim_customer.sql'. The query starts with a comment: '-- One row per customer, with the signup cohort month for new-vs-returning analysis.' followed by a 'select' statement. The 'select' statement lists columns: 'c.customer_id', 'c.customer_name', 'c.email', 'c.city', 'c.signup_date', and 'date_trunc('month', c.signup_date) as signup_month'. The 'from' clause is 'from {{ ref('stg_customers') }} c'. The editor has a dark theme and a sidebar on the right.

```
dim_customer.sql × dim_date.sql 1 × dim_product.sql 3 ×
atliq_gold > models > gold > dim_customer.sql
ψ Connect to MSSQL
1  -- One row per customer, with the signup cohort month for new-vs-returning analysis.
2  select
3      c.customer_id,
4      c.customer_name,
5      c.email,
6      c.city,
7      c.signup_date,
8      date_trunc('month', c.signup_date) as signup_month
9  from {{ ref('stg_customers') }} c
```

dim_customer: one row per customer with signup_month for cohorts

Build the Dimensions



```
dim_customer.sql × dim_date.sql 1 × dim_product.sql 3 ×
atliq_gold > models > gold > dim_product.sql
❏ Connect to MSSQL
1  -- One row per product, enriched with supplier cost (from the CSV source) and margin.
2  select
3      p.product_id,
4      p.product_name,
5      p.category,
6      p.unit_price,
7      s.supplier_cost,
8      (p.unit_price - s.supplier_cost) as unit_margin
9  from {{ ref('stg_products') }} p
10 left join {{ ref('stg_supplier_price_list') }} s
11      on p.product_id = s.product_id
```

dim_product: joins supplier cost and computes unit_margin

Build the Dimensions



```
dim_customer.sql x dim_date.sql 1 x dim_product.sql 3 x
atliq_gold > models > gold > dim_date.sql
❏ Connect to MSSQL
1  -- Calendar spine 2024-01-01 .. 2026-12-31 (covers order dates + earliest signups).
2  with days as (
3      select explode(
4          sequence(to_date('2024-01-01'), to_date('2026-12-31'), interval 1 day)
5      ) as date_day
6  )
7  select
8      cast(date_format(date_day, 'yyyyMMdd') as int) as date_key,
9      date_day,
10     day(date_day) as day,
11     month(date_day) as month,
12     date_format(date_day, 'MMMM') as month_name,
13     quarter(date_day) as quarter,
14     year(date_day) as year,
15     date_format(date_day, 'EEEE') as weekday_name,
16     case when dayofweek(date_day) in (1,7) then true else false end as is_weekend
17 from days
```

dim_date: generated calendar spine, 2024-01-01 to 2026-12-31

5

STEP 5

Build the Fact Table

WHAT

Wrote fact_sales at one row per order item, joining order_items to orders, computing gross_revenue = quantity x item_price, carrying date_key and status, with foreign keys to each dimension.

WHY

The grain (one row per order item) is the finest level, so reports can aggregate up but never lose detail. Key strategy: natural business keys (customer_id, product_id, date_key) are used rather than surrogate keys, since the source IDs are already clean unique integers and the dimensions are Type-1 (no history to version). Surrogate keys would be introduced if slowly-changing dimensions or multi-source integration were needed.

Build the Fact Table

```
fact_sales.sql 3 X
atliq_gold > models > gold > fact_sales.sql
ψ Connect to MSSQL
1  -- Grain: one row per order item. Measures: quantity, item_price, gross_revenue.
2  select
3      oi.order_item_id,
4      o.order_id,
5      o.customer_id,
6      oi.product_id,
7      o.order_date,
8      cast(date_format(o.order_date, 'yyyyMMdd') as int) as date_key,
9      oi.quantity,
10     oi.item_price,
11     (oi.quantity * oi.item_price) as gross_revenue,
12     o.status
13 from {{ ref('stg_order_items') }} oi
14 join {{ ref('stg_orders') }} o
15 on oi.order_id = o.order_id
```

fact_sales: order-item grain, date_key, gross_revenue = quantity x item_price



STEP 6

Add Data-Quality Tests

WHAT

Defined tests in schema.yml: unique and not_null on all keys, relationships from fact_sales to each dimension, accepted_values on status, and dbt_utils accepted_range on gross_revenue (≥ 0). Installed dbt_utils via dbt deps.

WHY

Tests turn the star schema's integrity rules into automated checks: no duplicate or orphaned keys, valid status domain, no negative revenue. A broken build fails loudly instead of shipping bad data.

Add Data-Quality Tests

```
(.venv) PS C:\de-for-da\capstone-project\dbt\atliq_gold> dbt test
15:59:39 Running with dbt=1.12.0
15:59:41 Registered adapter: databricks=1.12.4
15:59:43 Found 10 models, 16 data tests, 7 sources, 889 macros
15:59:43 Concurrency: 4 threads (target='dev')
15:59:43
15:59:46 1 of 16 START test accepted values fact_sales_status Placed Shipped Delivered Cancelled Returned [RUN]
15:59:46 2 of 16 START test dbt_utils accepted_range fact_sales_gross_revenue_True_0 .. [RUN]
15:59:46 3 of 16 START test not_null_dim_customer_customer_id ..... [RUN]
15:59:46 4 of 16 START test not_null_dim_date_date_key ..... [RUN]
15:59:48 2 of 16 PASS dbt_utils accepted_range fact_sales_gross_revenue_True_0 ..... [PASS in 1.88s]
15:59:48 1 of 16 PASS accepted values fact_sales_status Placed Shipped Delivered Cancelled Returned [PASS in 1.89s]
15:59:48 4 of 16 PASS not_null_dim_date_date_key ..... [PASS in 1.88s]
15:59:48 3 of 16 PASS not_null_dim_customer_customer_id ..... [PASS in 1.89s]
15:59:48 5 of 16 START test not_null_dim_product_product_id ..... [RUN]
15:59:48 6 of 16 START test not_null_dim_product_unit_price ..... [RUN]
15:59:48 7 of 16 START test not_null_fact_sales_customer_id ..... [RUN]
15:59:48 8 of 16 START test not_null_fact_sales_order_item_id ..... [RUN]
15:59:50 6 of 16 PASS not_null_dim_product_unit_price ..... [PASS in 1.72s]
15:59:50 5 of 16 PASS not_null_dim_product_product_id ..... [PASS in 1.72s]
15:59:50 7 of 16 PASS not_null_fact_sales_customer_id ..... [PASS in 1.70s]
15:59:50 8 of 16 PASS not_null_fact_sales_order_item_id ..... [PASS in 1.70s]
15:59:50 9 of 16 START test not_null_fact_sales_product_id ..... [RUN]
15:59:50 10 of 16 START test relationships fact_sales_customer_id_customer_id_ref_dim_customer_ [RUN]
15:59:50 11 of 16 START test relationships fact_sales_date_key_date_key_ref_dim_date_ [RUN]
15:59:50 12 of 16 START test relationships fact_sales_product_id_product_id_ref_dim_product [RUN]
15:59:51 9 of 16 PASS not_null_fact_sales_product_id ..... [PASS in 1.56s]
15:59:51 13 of 16 START test unique_dim_customer_customer_id ..... [RUN]
15:59:51 10 of 16 PASS relationships fact_sales_customer_id_customer_id_ref_dim_customer_ [PASS in 1.75s]
15:59:51 12 of 16 PASS relationships fact_sales_product_id_product_id_ref_dim_product [PASS in 1.75s]
15:59:51 11 of 16 PASS relationships fact_sales_date_key_date_key_ref_dim_date_ ..... [PASS in 1.76s]
15:59:52 14 of 16 START test unique_dim_date_date_key ..... [RUN]
15:59:52 15 of 16 START test unique_dim_product_product_id ..... [RUN]
15:59:52 16 of 16 START test unique_fact_sales_order_item_id ..... [RUN]
15:59:53 15 of 16 PASS unique_dim_product_product_id ..... [PASS in 1.78s]
15:59:53 14 of 16 PASS unique_dim_date_date_key ..... [PASS in 1.80s]
15:59:53 13 of 16 PASS unique_dim_customer_customer_id ..... [PASS in 2.07s]
15:59:53 16 of 16 PASS unique_fact_sales_order_item_id ..... [PASS in 1.77s]
15:59:53
15:59:53 Finished running 16 data tests in 0 hours 0 minutes and 10.73 seconds (10.73s).
15:59:54
15:59:54 Completed successfully
15:59:54
15:59:54 Done. PASS=16 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=16
(.venv) PS C:\de-for-da\capstone-project\dbt\atliq_gold>
```

dbt test: all 16 data tests PASS



STEP 7

Control Schema Naming (macro)

WHAT

By default, dbt's schema-naming concatenates the target and custom schema names (producing `gold_gold`). Added a `generate_schema_name` macro so models land cleanly in `atliq.gold`, the standard dbt practice for explicit schema control.

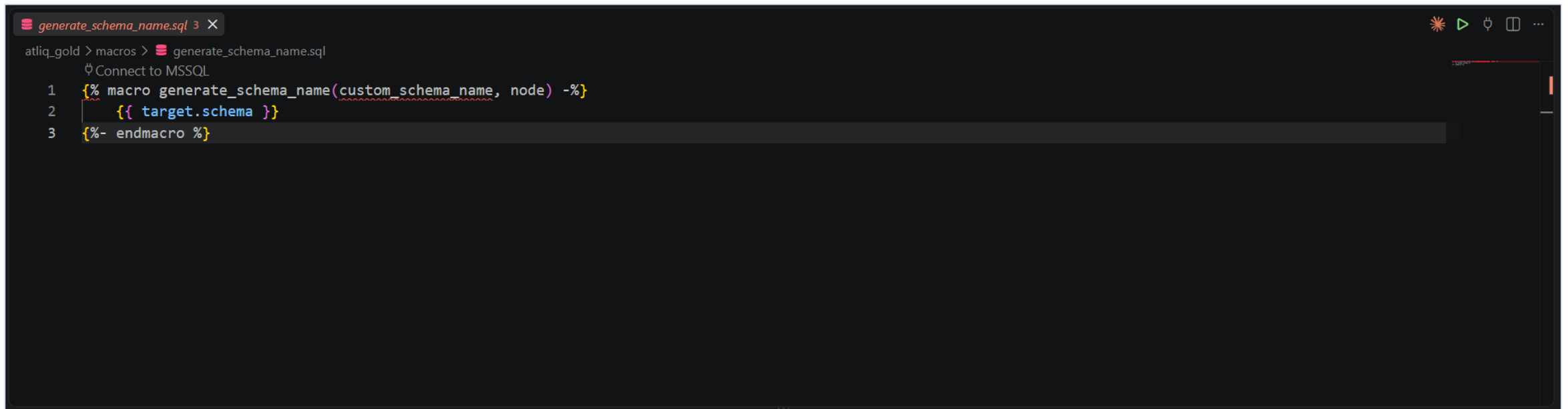
WHY

Controlling where models are created is a deliberate design choice, not a default to accept. Overriding `generate_schema_name` is the standard, documented way to make dbt build into exactly the schema you intend.

7

STEP 7 • SCREENSHOT

Control Schema Naming (macro)



The screenshot shows a SQL editor window with a tab titled 'generate_schema_name.sql 3'. The editor content is as follows:

```
atliq_gold > macros > generate_schema_name.sql
ψ Connect to MSSQL
1  {% macro generate_schema_name(custom_schema_name, node) -%}
2      {{ target.schema }}
3  {% endmacro %}
```

The editor has a dark theme and includes standard icons for running, debugging, and other SQL-related actions in the top right corner.

generate_schema_name macro returning target.schema



STEP 8

Build and Test (dbt build)

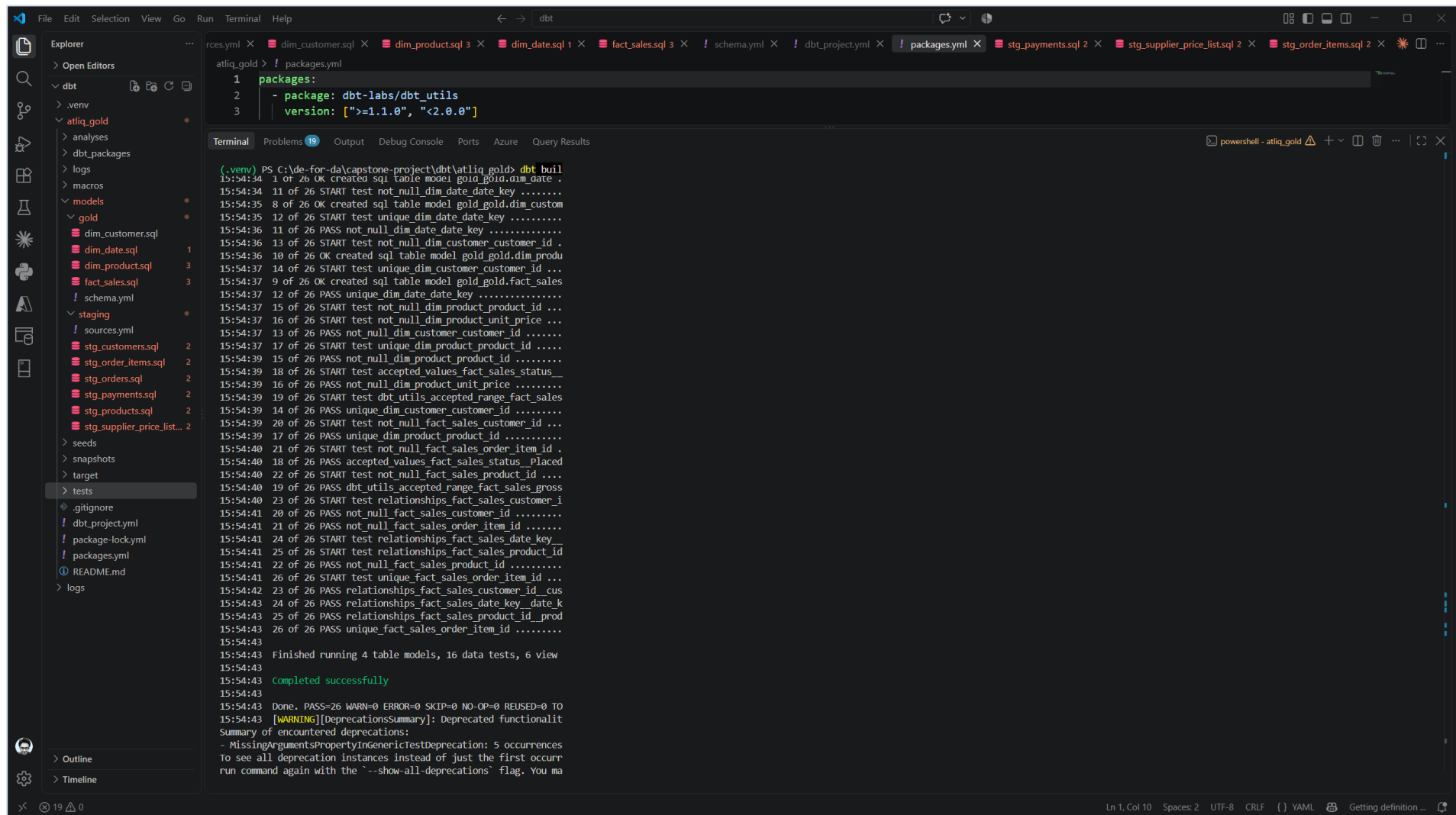
WHAT

Ran dbt build: 6 staging views + 4 gold tables built, and all 16 data tests passed. Done. PASS=26 WARN=0 ERROR=0.

WHY

dbt build runs models and tests together in dependency order. Every relationship test passing means the fact table's keys all resolve to their dimensions, referential integrity, proven.

Build and Test (dbt build)



dbt build: 26 of 26 models and tests passed, Completed successfully



STEP 9

Verify the Gold Layer

WHAT

Confirmed atliq.gold now holds all 10 objects: 4 tables (dim_customer, dim_product, dim_date, fact_sales) and 6 staging views.

WHY

The populated gold schema is the M4 deliverable and the serving layer Fabric reads in M6. The clean before/after (empty gold, then 10 objects) shows the transformation landing correctly.

Verify the Gold Layer

▶

✓ Just now (1s)

1

SQL

🗑️

🔗

📐

⋮

```
%sql
SHOW TABLES IN atliq.gold;
```

> [See performance \(1\)](#) Optimize

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [database: string, tableName: string ... 1 more field]`

Table ▾

+

🔍

🔧

📄

	database	tableName	isTemporary
1	gold	dim_customer	false
2	gold	dim_date	false
3	gold	dim_product	false
4	gold	fact_sales	false
5	gold	stg_customers	false
6	gold	stg_order_items	false
7	gold	stg_orders	false
8	gold	stg_payments	false
9	gold	stg_products	false
10	gold	stg_supplier_price_li...	false

⬇️ ▾

10 rows | 0.831s runtime

Refreshed Now

📘

This result is stored as `_sqldf` and can be used in other [Python](#) and [SQL](#) cells.

[Send feedback](#)

SHOW TABLES IN atliq.gold: 10 objects (4 tables, 6 staging views)

Milestone 4 Complete

What this milestone delivered

Star schema in Gold

fact_sales at order-item grain, surrounded by dim_customer, dim_product, dim_date. The analytics-serving shape Fabric reads.

Transformations in dbt

Supplier margin, signup cohorts, a generated date spine, and a single revenue definition, all as version-controlled dbt models.

Tested and trustworthy

16 tests: unique and not-null keys, fact-to-dimension referential integrity, valid status, non-negative revenue. All passing.

Star schema objects

1

fact table

3

dimensions

6

staging views

16

tests passed

0

errors

Engineering decisions: overrode dbt's default schema-naming with a generate_schema_name macro for explicit control over where models are built; kept credentials in a profiles file outside the repo, referenced by environment variables for the committed project.